

Copyright vs. License

Owning a book's content

- You are the author and copy right holder
- It is copyrighted by default
- You have the only permission to grant rights on your copyrighted material to someone

Buying a book

- Buying a book is not buying its right to make copies (copyright).
- If you need to buy or just grant copyright, you need a license...
- An agreement that you are allowed to make, distribute copies, alter content, use it for your own work, re-license it with a different license, or you are forced to always use the same license in derived work, must include all authors, etc.



Copyright vs. License

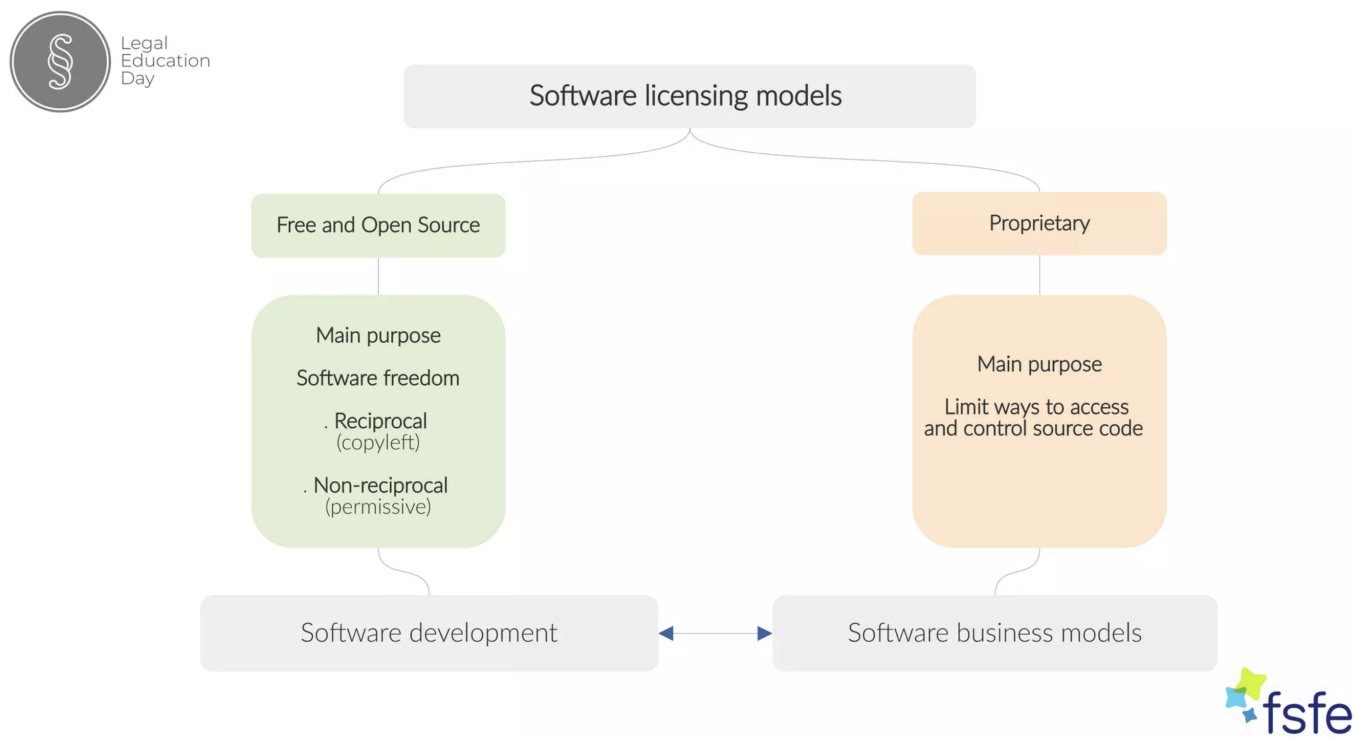
Legalese (in simple terms)

1. **Copyright** is the legal term used to declare and prove who owns the intellectual property (the code, text, etc.).
 - Right of attribution
 - Economic rights: Right of reproduction, broadcasting, communication,...
 - Right of integrity
 - Right of adaption / translation
2. **Licensing** is the legal term used to describe the terms under which people are allowed to use the copyrighted material.
3. **Copyright holder** is who has the rights to copy, redistribute, etc.
4. **Author** is who wrote the code, book, ...
 - An AUTHOR is not always the COPYRIGHT HOLDER

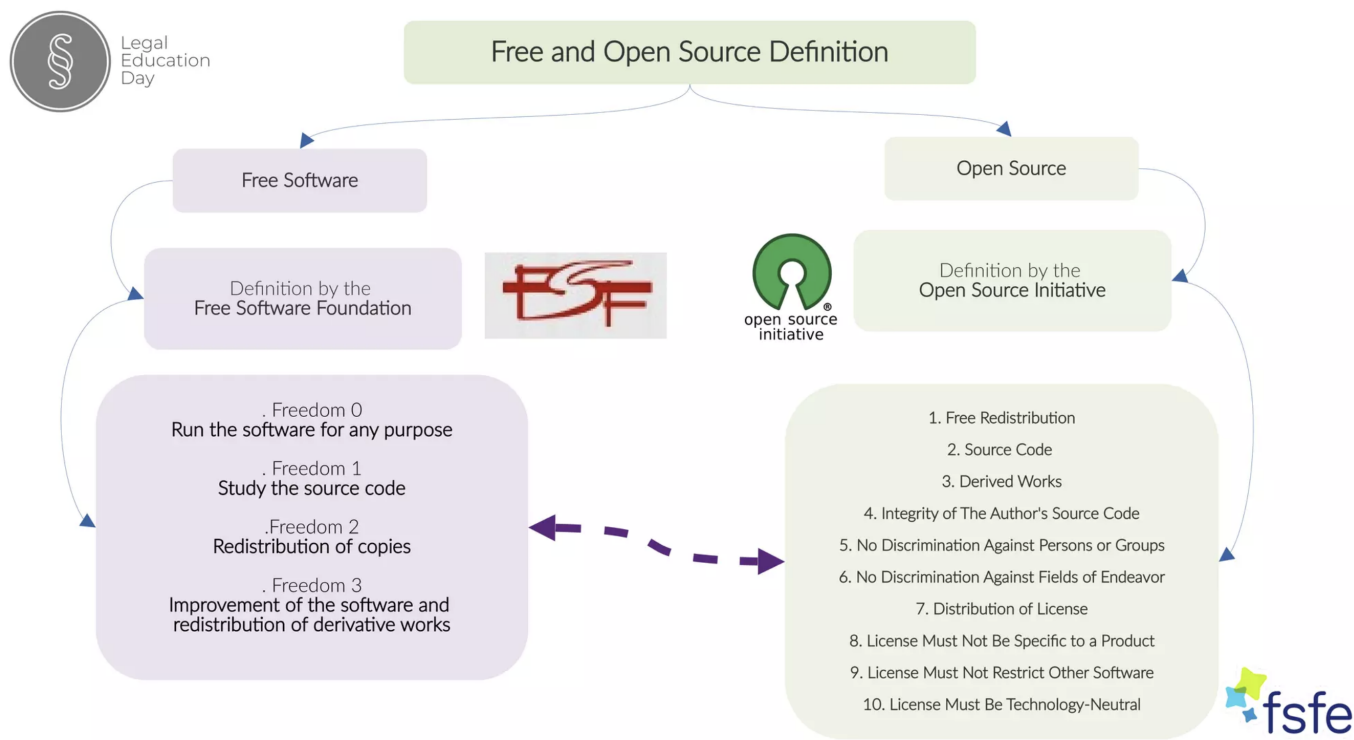


Software licensing models

Free and Open Source vs. Proprietary Licenses



Open-Source License types



License: permissive, copyleft, strong copy-left, network copy-left, proprietary

Some examples

Permissive	Weak copyleft	Strong copyleft licenses	Network Copyleft
Allow any kind of usage of code. Permissive licenses permit software to become proprietary.	Prevent the software component (often a software library) from becoming proprietary, yet permit it to be part of a larger proprietary program.	Prevent the software from becoming proprietary.	Prevent the software from becoming proprietary on network enviroment, triggering copyleft not only by distributing but also using it.
Most popular liceses (SPDX code)			
MIT (MIT) A short and simple permissive license with conditions only requiring preservation of copyright and license notices. Licensed works, modifications, and larger works may be distributed under different terms and without source code.	LGPL (LGPL-3.0) The license allows developers and companies to use and integrate a software component released under the LGPL into their own (even proprietary) software without being required by the terms of a strong copyleft license to release the source code of their own components. The use of this license is discouraged by the FSF.	GPL v3 (GPL-3.0) The most popular Free Software license. Permissions of this strong copyleft license are conditioned on making available complete source code of licensed works and modifications, which include larger works using a licensed work, under the same license. Copyright and license notices must be preserved. Contributors provide an express grant of patent rights.	Affero GPL v3 (AGPL-3.0) Based on GPL v3, is dedicated to software used over a network (SaaS for instance). This provision requires that the full source code be made available to any network user.
Apache 2.0 (Apache-2.0) A permissive license whose main conditions require preservation of copyright and license notices. Contributors provide an express grant of patent rights. Licensed works, modifications, and larger works may be distributed under different terms and without source code.	Mozilla Public License 2.0 (MPL 2.0) A weak copyleft license, characterized as a middle ground between permissive free software licenses and the GNU General Public License (GPL), that seeks to balance the concerns of proprietary and open source developers. a larger work using the licensed work may be distributed under different terms and without source code for files added in the larger work.		



The MIT License

 Legal Education Day

Non-Reciprocal Licenses (permissive)

MIT License

```
Copyright (c) <year> <copyright holders>

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

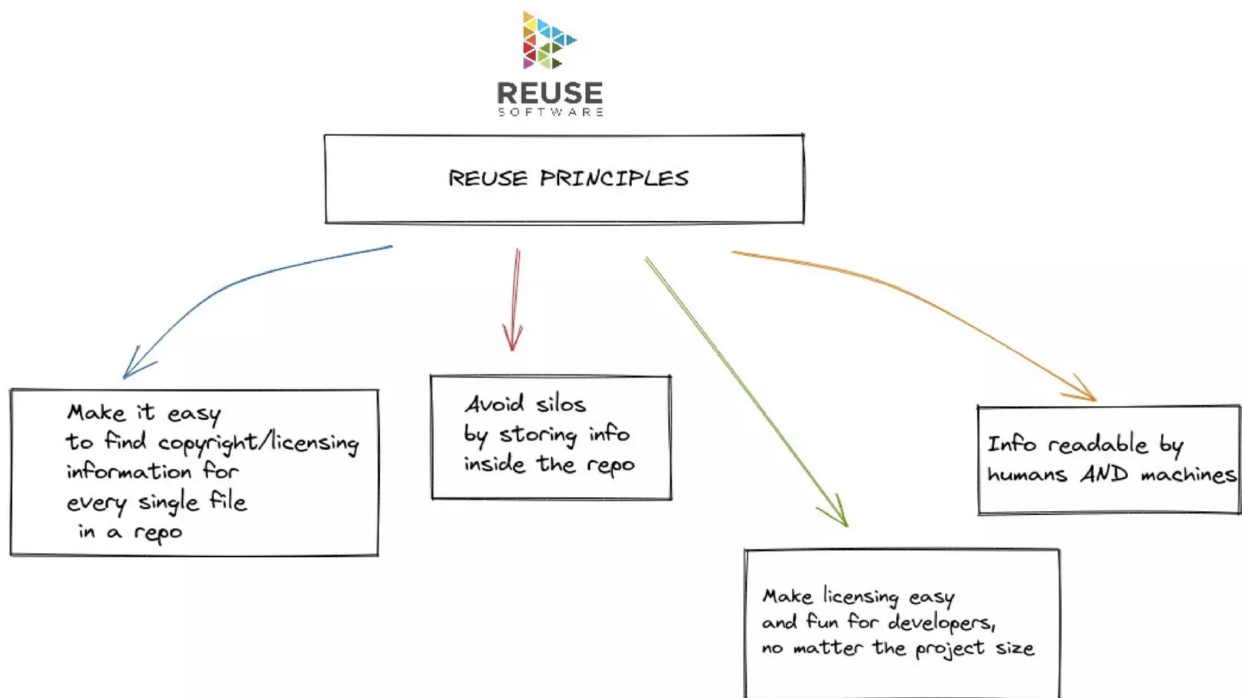
Copyright grants

Notice obligation



What a boring job! Tools for developers

REUSE



<https://reuse.software>

Show me the code, Luke!

```
# Install REUSE
pip install --user reuse

# Get help
reuse

# Lint the whole project, do we have any problems in legal terms?
reuse lint
```

Fix the problems

This means, we need to give a license information to any file in the project. This can be done in two ways:

1. Write a license header to the files, or attach a .license file if the target is binary
2. Write a license annotations file called **REUSE.toml**

WAY #1: Attach license information as header in each file

Manual or with **reuse**.

```
/*
 * SPDX-FileCopyrightText: 2024 AboutBits <info@aboutbits.it>
 *
 * SPDX-License-Identifier: MIT
 */

#include <stdio.h>
int main()
{
    printf("Hello, World!");
    return 0;
}
```

- No need to write the whole or a part of the license text into each file. A header is good enough. Preferable SPDX, because it is a recognized standard, machine-readable, and used in various tools.
- License year, copyright holder's name, and e-mail
- License identifier
- The license text will be stored in the **LICENSES** folder

```
reuse annotate --copyright "AboutBits <info@aboutbits.it>" --license "MIT"
src/main.c Makefile
```

```
reuse annotate --copyright "AboutBits <info@aboutbits.it>" --license "CC0-1.0"
README.md .gitignore docs/*
```

```
reuse annotate --copyright "AboutBits <info@aboutbits.it>" --license="CC-BY-4.0"
--force-dot-license img/cat.jpg img/dog.jpg
```

No let's download all licenses into the **LICENSES** directory.

```
reuse download --all
```

WAY #2: Write a toml file to define which license applies to what file

```
version = 1
SPDX-PackageName = "reuse-example"
SPDX-PackageSupplier = "Peter Moser <peter.moser@aboutbits.it>"
SPDX-PackageDownloadLocation = "https://github.com/some/awesome-project"

[[annotations]]
path = ["src/**", "Makefile"]
precedence = "aggregate"
SPDX-FileCopyrightText = "(c) 2021 AboutBits <info@aboutbits.it>"
```

```
SPDX-License-Identifier = "MIT"

[[annotations]]
path = ["README.md", ".gitignore", "docs/**"]
precedence = "aggregate"
SPDX-FileCopyrightText = "(c) 2021 AboutBits <info@aboutbits.it>"
SPDX-License-Identifier = "CC0-1.0"

[[annotations]]
path = "img/**/*.jpg"
precedence = "aggregate"
SPDX-FileCopyrightText = "(c) 2021 AboutBits <info@aboutbits.it>"
SPDX-License-Identifier = "CC-BY-4.0"
```

Finally, let's run the test again!

```
# Lint the whole project, do we have any problems in legal terms?
reuse lint
```

Dual licensing

MIT or LGPL for example

NB: SPDX identifiers can be concatenated by a boolean expression (ex., OR or AND). Haven't seen that much...

<https://opensource.stackexchange.com/a/8813>

If I decide to sell you a car, I can offer to do so for (say) a thousand pounds, or if I choose I might say to you "You can have my car for either a thousand pounds or your motorcycle in exchange". If you decide you want my car it's up to you to decide which condition to honour: the money or the motorcycle.

That is the essence of dual licensing. I offer to give you a copy of some code, plus the rights to do certain things with it, and in exchange I require that you agree to honour either this set of conditions here (the MIT licence), or this other set of conditions here (the GNU GPL). If you want the code and the rights, you must decide which set of conditions you will accept. In theory you should probably decide then and there, though in practice it makes no difference until you want to do something, probably redistribution, where the two set of requirements differ.

If you decide to accept the code under MIT, then the added restrictions of the GPL no longer apply to your copy; if you decide to accept it under GPL, then the absence of those restrictions from the MIT licence no longer matters with respect to your copy. That's why any contradiction between the two doesn't matter in this case.

Edit: you have pointed us to the code in question, for which many thanks. Reading eg this file, the author writes "Dual licensed under the MIT or GPL Version 2 licenses". So yes, you can choose which one of those licences you receive it under, and once you have done so, you need not worry about the requirements of the one you didn't choose.

Inbound vs. outbound license

- inbound (from contributors)
- outbound (to other contributors and users)

For the vast majority of open source projects, an open source license implicitly serves as both the **inbound** and **outbound** license.

If your project is on GitHub, the GitHub Terms of Service make “inbound=outbound” the explicit default.

If needed, create a Contributor License Agreement (CLA)

But a lot of work, and may limit contributions:

We have eliminated the CLA for Node.js. Doing this lowers the barrier to entry for Node.js contributors thereby broadening the contributor base.

— @bcantrill, “Broadening Node.js Contributions”

Copyrights and trademarks defined

Copyrights and trademarks protect distinct creations:

- copyrights protect creative works
- trademarks apply to brand names, phrases, and logos

Appendix

GitHub CC (Continuous Compliance)

<https://github.com/marketplace/actions/reuse-compliance-check>

Help for devs: <https://reuse.software/dev/>

```
# SPDX-FileCopyrightText: 2022 Free Software Foundation Europe e.V.
<https://fsfe.org>
#
# SPDX-License-Identifier: CC0-1.0

name: REUSE Compliance Check

on: [push, pull_request]

jobs:
  reuse-compliance-check:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: REUSE Compliance Check
        uses: fsfe/reuse-action@v5
```

PRE-COMMIT HOOK

```
# SPDX-FileCopyrightText: 2020 Free Software Foundation Europe e.V.
# SPDX-License-Identifier: CC0-1.0
repos:
  - repo: https://github.com/fsfe/reuse-tool
    rev: v5.0.2
    hooks:
      - id: reuse
```

Some links

- <https://fsfe.org/freesoftware/legal/faq.de.html>
- <https://opensource.guide/legal/>
- <https://choosealicense.com/>
- <https://spdx.org/licenses/>
- <https://creativecommons.org/choose/>
- <https://www.sfscon.it/talks/what-are-some-common-free-software-licenses/>

Italian based Law Firm fighting for Open Source License rights

The first AGPL compliance case settled in an Italian Court: a tale of compliance, license compatibility and source code availability

- <https://av.tib.eu/media/13945>
- <https://array.eu>

Thank you for your attention!
